

P1659R0

starts_with and ends_with

Published Proposal, 2019-06-17

This version:

<https://wg21.link/p1659>

Author:

[Christopher Di Bella](#)

Audience:

LEWG, SG18

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This proposal seeks to add `std::ranges::starts_with` and `std::ranges::ends_with`, which would work on arbitrary ranges, and also answer questions such as "are the starting elements of ``r1`` *less than* the elements of ``r2``?" and "are the final elements of ``r1`` *greater than* the elements of ``r2``?". It also proposes a change to SD-8.

Table of Contents

1 Motivation

1.1 Example usage

2 Proposed changes to SD-8

3 Proposed changes to C++23

3.1 25.5.14 Starts with [alg.starts_with]

3.2 25.5.15 Ends with [alg.ends_with]

4 Reference implementation

5 Acknowledgements

References

Informative References

§ 1. Motivation

C++20 introduces `basic_string_view::starts_with`, `basic_string_view::ends_with`, `basic_string::starts_with`, and `basic_string::ends_with`. Both `basic_string` and `basic_string_view` are peculiar container-like types, with many member functions that duplicate algorithm functionality with minor interface changes (e.g. `compare`, `copy`, `find`, `rfind`, `find_first_of`, etc.). [P0457] §5.1 notes that the decision to add `starts_with` and `ends_with` as member functions was because (a) it is consistent with the aforementioned member functions, and (b) that P0457 agrees with [N3609] in that `starts_with(haystack, needle)` is ambiguous with `starts_with(needle, haystack)`. It should be noted that neither N3609, nor P0457 identified whether or not they were talking about non-member functions that only operate on string types, or if they were discussing an algorithm, but the LEWG minutes for P0457 reveal some LWEG interest in making them algorithms.

Although there is prior art with respect to `basic_string`'s member functions, the author expresses concern that our string types have a large set of member functions that either duplicate the algorithms or are directly incompatible with them, and

thus limit the amount of composition that's possible. Templates are one of C++'s greatest strengths, and with iterators, we have an extremely extensible and powerful generic programming model. We should take advantage of this model wherever possible to ensure that we do not paint ourselves into a corner with a single type.

At present, it isn't *immediately* possible to query whether or not any range -- other than a standard string type -- is prefixed or suffixed by another range. To do so, one must use `mismatch` or `equal`, and at least in the case of `ends_with`, `ranges::next` (C++20).

Before-and-after table	
C++20	
<pre>auto some_ints = view::iota(0, 50); auto some_more_ints = view::iota(0, 30); if (ranges::mismatch(some_ints, some_more_ints).in2 == end(some_more_ints)) { // do something }</pre>	<pre>auto some_ints = view::iota(0, 50); auto some_more_ints = view::iota(0, 30); if (ranges::starts_with(some_ints, some_more_ints)) { // do something }</pre>
<pre>auto some_ints = view::iota(0, 50); auto some_more_ints = view::iota(0, 30); { auto some_ints_tail = subrange(next(begin(some_ints), distance(some_more_ints), end(some_ints)), end(some_ints)); if (not equal(some_ints_tail, some_more_ints)) { // do something } }</pre>	<pre>auto some_ints = view::iota(0, 50); auto some_more_ints = view::iota(0, 30); if (not ranges::ends_with(some_ints, some_more_ints)) { // do something }</pre>

It is interesting to note that, since `starts_with` and `ends_with` can be implemented using `mismatch`, we are able to query more than just "are the first n elements of range one the same as the entirety of range two?": we're also able to query "are the first n elements of range one all greater than the entirety of range two?", where n is the distance of the second range. See §1.1 for examples. This is something that the string-based member functions are not able to do, although the author acknowledges that text processing may not require this functionality.

Concerns were outlined in both N3609 and P0457 about the ambiguity of whether we are performing `starts_with(haystack, needle)` or `starts_with(needle, haystack)`. There is prior art in the algorithms library that makes the first range the subject of the operation: `mismatch`, `equal`, `search`, `find_first_of`, and `lexicographical_compare` all take the form `algorithm(haystack, needle)`, so the author remains unconvinced about the ambiguity.

§ 1.1. Example usage

```
auto script = u8"OBI-WAN: Hello, there!\n"
              u8"GENERAL GRIEVOUS: General Kenobi, you are a bold one."sv;
namespace ranges = std::ranges;
ranges::starts_with(script, u8"OBI-WAN"sv);           // returns true
ranges::starts_with(script, u8"ABCDEFGF"sv);           // returns false
ranges::starts_with(script, u8"ABCDEFGF"sv, ranges::less); // returns true

namespace view = ranges::view;
ranges::ends_with(view::iota(0, 50), view::iota(30) | view::take(20)); // returns true
ranges::ends_with(view::iota(0, 50), view::iota(30) | view::take(50)); // returns false
ranges::ends_with(view::iota(0, 50), view::iota(-50, -40), ranges::greater); // returns true
```

§ 2. Proposed changes to SD-8

In response to the concerns outlined in the motivation section of this document, the author would like to request that LEWG consider discussing adopting in SD-8, a policy of ensuring that options for algorithms are preferred when a proposal to add a member function to a container-like type is considered.

§ 3. Proposed changes to C++23

Add the following text to [algorithm.synopsis]:

```
...
template<class ForwardIterator, class Searcher>
constexpr ForwardIterator
    search(ForwardIterator first, ForwardIterator last, const Searcher& searcher);

+ namespace ranges {
+     // [alg.starts_with], starts_with
+     template<InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
+             class Comp = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
+         requires IndirectlyComparable<I1, I2, Comp, Proj1, Proj2>
+         constexpr bool starts_with(I1 first1, S1 last1, I2 first2, S2 last2, Comp comp = {},
+                                     Proj1 proj1 = {}, Proj2 proj2 = {});
+     template<InputRange R1, InputRange R2, class Comp = ranges::equal_to, class Proj1 = identity,
+             class Proj2 = identity>
+         requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Comp, Proj1, Proj2>
+         constexpr bool starts_with(R1&& r1, R2&& r2, Comp comp = {}, Proj1 proj1 = {},
+                                     Proj2 proj2 = {});
+     // [alg.ends_with], ends_with
+     template<ForwardIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
+             class Comp = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
+         requires IndirectlyComparable<I1, I2, Comp, Proj1, Proj2>
+         constexpr bool ends_with(I1 first1, S1 last1, I2 first2, S2 last2, Comp comp = {},
+                                    Proj1 proj1 = {}, Proj2 proj2 = {});
+     template<ForwardRange R1, InputRange R2, class Comp = ranges::equal_to, class Proj1 = identity,
+             class Proj2 = identity>
+         requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Comp, Proj1, Proj2>
+         constexpr bool ends_with(R1&& r1, R2&& r2, Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
+ }

// [alg.modifying.operations], mutating sequence operations
// [alg.copy], copy
...
```

Add the following two sections to [alg.nonmodifying]:

§ 3.1. 25.5.14 Starts with [alg.starts_with]

```
namespace ranges {
    template<InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
            class Comp = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires IndirectlyComparable<I1, I2, Comp, Proj1, Proj2>
        constexpr bool starts_with(I1 first1, S1 last1, I2 first2, S2 last2, Comp comp = {},
                                   Proj1 proj1 = {}, Proj2 proj2 = {});
    template<InputRange R1, InputRange R2, class Comp = ranges::equal_to, class Proj1 = identity,
            class Proj2 = identity>
        requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Comp, Proj1, Proj2>
        constexpr bool starts_with(R1&& r1, R2&& r2, Comp comp = {}, Proj1 proj1 = {},
                                   Proj2 proj2 = {});
}
```

1. Equivalent to:

```
return ranges::mismatch(first1, last1, first2, last2, comp, proj1, proj2).in2 == last2;
```

2. Complexity: If the types of `first1`, `last1`, `first2`, and `last2` pairwise model `SizedSentinel` and `last1 - first1 != last2 - first2`, then no applications of the corresponding predicate and each projection; otherwise, at most $\min(\text{last1} - \text{first1}, \text{last2} - \text{first2})$ applications of the corresponding predicate and projections.

§ 3.2. 25.5.15 Ends with [alg.ends_with]

```
template<ForwardIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
        class Comp = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
    requires IndirectlyComparable<I1, I2, Comp, Proj1, Proj2>
    constexpr bool ends_with(I1 first1, S1 last1, I2 first2, S2 last2, Comp comp = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});
template<ForwardRange R1, InputRange R2, class Comp = ranges::equal_to, class Proj1 = identity,
        class Proj2 = identity>
    requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Comp, Proj1, Proj2>
    constexpr bool ends_with(R1&& r1, R2&& r2, Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

1. Equivalent to:

```
const auto first = subrange{first1, last1};
const auto second = subrange{first2, last2};
return ranges::equal(first | view::drop(ranges::distance(second)), second, comp, proj1, proj2);
```

2. Complexity: If the types of `first1`, `last1`, `first2`, and `last2` pairwise model `SizedSentinel` and `last1 - first1 != last2 - first2`, then no applications of the corresponding predicate and each projection; otherwise, at most $\min(\text{last1} - \text{first1}, \text{last2} - \text{first2})$ applications of the corresponding predicate and projections.

§ 4. Reference implementation

Both `[starts_with]` and `[ends_with]` have been implemented in range-v3.

§ 5. Acknowledgements

The author would like to thank Arien Judge for reviewing the proposal, Johel Ernesto Guerrero Peña for providing an implementation for `ends_with`, and Eric Niebler for merging the respective pull requests to range-v3.

§ References

§ Informative References

[ENDS_WITH]

Johel Ernesto Guerrero and Eric Niebler. [ranges::ends_with](https://git.io/fjzq0). URL: <https://git.io/fjzq0>

[N3609]

Jeffrey Yasskin. [string_view: a non-owning reference to a string, revision 3](https://wg21.link/n3609). URL: <https://wg21.link/n3609>

[P0457]

Mikhail Maltsev. [String Prefix and Suffix Checking](https://wg21.link/p0457). URL: <https://wg21.link/p0457>

[STARTS_WITH]

Christopher Di Bella and Eric Niebler. [ranges::starts_with](https://git.io/fjzqR). URL: <https://git.io/fjzqR>